

GLM alapú többváltozós statisztikai modellek biztosítási alkalmazásokkal

Vakhal Péter

2026-01-01

Table of contents

Előszó	4
A könyv (tevezett) felépítése:	4
Miről nem fog szólni a kurzus és a könyv?	5
A szerzőről	5
1 Bevezetés az R környezetbe és az adatkezelésbe	6
2 Miről lesz szó a fejezetben?	7
3 Az R programnyelv és az RStudio	8
3.1 R történelem	8
3.2 RStudio - karmester az R-hez	9
3.2.1 Az RStudio felépítése	9
3.3 A workspace használata, kódok írása	10
3.4 Csomagok, package-ek telepítése, használata	11
3.5 Mesterséges intelligencia és az R	12
4 R alapok	13
4.1 Műveletek változókkal	13
4.1.1 Változó definiálása	13
4.1.2 Indexálás	14
4.1.3 Vektorizáció	16
4.2 Hiányzó adatok és kezelésük	16
4.3 Alapstatisztikák	17
4.4 Faktorok	18
4.5 Adattáblák (data frames)	19
5 Adatok importálása és adatmanipuláció	22
5.1 Adatok importálása	22
5.2 Adatmanipuláció a <code>dplyr</code> csomag segítségével	23
5.2.1 <code>select()</code>	23
5.2.2 <code>filter()</code>	26
5.2.3 <code>mutate()</code>	28
5.2.4 <code>group_by</code> és <code>summarize</code>	29
5.2.5 <code>arrange()</code>	30
5.2.6 <code>xtabs()</code> , <code>table()</code> és egyéb keresztábrás megoldások	31

6	Feltáró adatelemzés és adatvizualizáció	32
7	Miről lesz szó a fejezetben?	33
8	Bevezetés a következtetéselméleti vizsgálatokba	34
8.1	Hipotézisek folytonos változók esetén (univariáns)	34
8.1.1	Adattípusok leíró statisztikája	35
8.1.2	Középértékek	38
8.1.3	Diszperzitási mutatók	39
8.1.4	Kvantilisok	39
8.1.5	Gyakoriságok	40
8.2	Alapvető statisztikai próbák	41
8.2.1	Hipotézisvizsgálat során elkövethető hibák	41
8.2.2	Paraméteres próbák	42
8.3	Hipotézisek diszkrét változók esetén (univariáns)	47
9	Adatvizualizáció és ggplot2 alapok	48
9.1	Eloszlások ábrázolása	48
9.2	Asszociáció ábrázolása	51
9.3	Vegyes kapcsolat ábrázolása	53
9.4	Korreláció ábrázolása	54

Előszó

Tisztelt Olvasó!

Ez az e-könyv a *Posta Biztosító* munkatársai számára készült és elsősorban a **Többváltozós statisztika** R programnyelven című tanfolyam támogatását szolgálja. A könyv részletesen tárgyalja az R programnyelv alapjait, azonban feltételezi bizonyos matematikai, statisztikai alaptudás (lineáris algebra, matematikai statisztika, mértékelmélet, valószínűesszámitás) meglétét. A tanfolyam célja, hogy megismertesse a hallgatókat az általánosított lineáris modellek (GLM) elméletével valamint a függvényapproximáció során alkalmazott diagnosztikai eljárásokkal, különös tekintettel a biztosítási modellezés során leggyakrabban előforduló eljárásokra. A kurzus során a lineáris és frekvencia valószínűségi felfogáson alapuló függvényillesztési és diagnosztikai eljárásokat tárgyaljuk, csupán az utolsó fejezet tartalmaz kitékintést a nemlineáris valamint a bayes-i esetekre.

A könyv nélkülözi a nem megkerülhetetlen matematikai levezetéseket, és elsősorban biztosítási esettanulmány alapokra épít. Az érdeklődő Olvasó azonban az ajánlott irodalomból teljes körűen tájékozódhat a tárgyalt módszerről. Az irodalmakat minden fejezet végén feltüntetjük.

Kérdés esetén állok rendelkezésükre: vakhal.peter@gmail.com

Jó munkát kívánok!

Bécs - Budapest, 2026. július 16.

Vakhal Péter

A könyv (tevezett) felépítése:

1. Bevezetés az R környezetbe és az adatkezelésbe
2. Exploratív elemzés és adatvizualizáció
3. Bevezetés a GLM elméletébe és az exponenciális eloszláscsalád alapjaiba
4. Faktorok, kontrasztok kódolása, elemzése GLM modellekben
5. Interakciók és bevezetés a nemlinearitásba GLM modellekben
6. Kárgyakoriság modellezése és a kitettség kezelése
7. Kárnagyság és a díj modellezése
8. Modellvalidáció és diagnosztika

9. Biztosítás specifikus témák (IBNR, kárfejlődési háromszög, bónusz-malusz, időfüggő kovariánsok stb.)
10. Kitekintés a GAM (nemlinearitás) és bayes-i modellekre

Miről nem fog szólni a kurzus és a könyv?

A többváltozós vagy többdimenziós statisztika egy rendkívül széles intervallumot magában foglaló tudományterület, aminek saját elméleti keretrendszere van, bár kétségkívül sokat merít az egyváltozós módszerek tárházából. Ezeket egy kurzus alkalmával lehetetlen lenne mind átvenni, de a teljesség igénye nélkül megemlíjtük őket.

- Többdimenziós következtetéselmélet
- Dimenziócsökkentési eljárások
- Szegmentációs, partíciós eljárások - felügyelő nélkül (pl.: klaszter)
- Szegmentációs, partíciós eljárások - felügyelővel (pl.: döntési fák)
- Topologikus skálázás (pl.: MDS)
- Hálózatok, gráfok
- Többváltozós idősorok (pl.: VAR modellek)

A szerzőről

Dr. Vakhal Péter, egyetemi adjunktus. [Doktori fokozatát](#) 2022-ben szerezte a Budapesti Corvinus Egyetemen. 2011 és 2026 között az egyetem Operációkutatás és Aktuáriustudományok Tanszékének munkatársa volt, másfél évtizednyi oktatói, kutatói tapasztalattal rendelkezik többváltozós statisztika területen. 2026 augusztustól a Pannon Egyetem Kvantitatív Módszerek Intézeti Tanszék egyetemi adjunktusa. Emellett 2006-tól dolgozik kutatóként a Kopint-Tárki Konjunktúratkutató Intézetben. Egy gyermek édesapja. [Linkedin profil](#)

1 Bevezetés az R környezetbe és az adatkezelésbe

2 Miről lesz szó a fejezetben?

A fejezetben megismertetjük az olvasót az R programnyelv alapjaival valamint a legegyszerűbb adatkezelési technikáival. Fontos tudni, hogy az R nem adatkezelő szoftver, hanem egy kifejezetten statisztikai elemzésre kifejlesztett, nyílt forráskódú, tudományos közösség által fenntartott programozási nyelv. Adatkezelési lehetőségei ezért erősen korlátozottak, és bár minden végrehajtható, gyakran egyszerűbb egy táblázatkezelő szoftverben (pl.: Microsoft Excel) megoldani bizonyos műveleteket. Látni fogjuk, hogy hatványozottan igaz ez az adattisztításra, mivel az R nem rendelkezik beépített vizuális adatkezelő alkalmazással. A fejezet végén az Olvasó, kiegészítve az előadás során hallottakkal, képes lesz adatok betöltésére, azok manipulációjára, lekérdezések, kimutatások készítésére, valamint alapvető statisztikai becslések készítésére.

3 Az R programnyelv és az RStudio

3.1 R történelem

Az R programnyelv a világ egyik legelterjedtebb programozás nyelve statisztikai elemzői, kutatói körökben. Mindez elsősorban annak köszönhető, hogy a nyílt forráskódot a tudományos közösség folyamatosan fejleszti, bővíti. A nyelv fókusza minden tudományterületre kiterjed a bölcsészettudományoktól kezdve, a természettudományokon át a társadalomtudományokig. A jelenleg szabadon elérhető modulok, csomagok (*“package”* - algoritmusokat, rutinokat tartalmazó függvények) száma több tízezer a *“kínálat”* pedig folyamatosan bővül, így minden diszciplína megtalálhatja a számára releváns függvényeket.

Kis R történelem

Az R programnyelv története az 1990-es évek elejére nyúlik vissza, amikor Ross Ihaka és Robert Gentleman az Aucklandi Egyetemen megalkották az S nyelv nyílt forráskódú dialektusaként. Az elmúlt évtizedekben az R a statisztikai modellezés, az adatelemzés és a tudományos igényű vizualizáció globális, de facto standardjává nőtte ki magát. Az iparban – különösen a gyógyszerkutatásban (biostatistika), a pénzügyi kockázatelemzésben, a makrogazdasági modellezésben és az akadémiai szférában – betöltött szerepe megkerülhetetlen, amit elsősorban a reprodukálható kutatást támogató filozófiájának és a CRAN ökoszisztéma páratlanul gazdag csomagtárának köszönhet.

Összehasonlítás más nyelvekkel

Bár a modern adattudományi projektekben gyakran merül fel a nyelvválasztás kérdése, az R markánsan elkülönül alternatíváitól:

- **R vs. Python:** Míg a Python egy általános célú szoftverfejlesztő nyelv, amely kiválóan integrálható éles produkciós környezetekbe és dominál a mély tanulás (Deep Learning) területén, addig az R-t kifejezetten adatokkal dolgozó szakemberek tervezték adatokkal dolgozó szakembereknek. Az R „statisztikai gondolkodásmódja”, az adatkeretek natív kezelése és a *tidyverse* nyújtotta elegáns adatmanipuláció páratlan hatékonyságot biztosít a feltáró elemzések során.
- **R vs. Matlab:** A Matlab kiváló, robusztus mérnöki és mátrixalapú szimulációs keretrendszer, ám zárt forráskódú és magas licencköltségekkel jár. Ezzel szemben az R teljesen ingyenes, és nyílt közösségének köszönhetően a legújabb elméleti statisztikai és ökonometria módszerek szinte azonnal elérhetővé válnak benne

csomagok formájában.

- **R vs. C/C++:** A C-család tagjai rendkívül gyors, alacsony szintű nyelvek, amelyek optimálisak rendszerprogramozásra, de fejlesztési idejük és komplexitásuk miatt alkalmatlanok az adatok gyors, interaktív elemzésére. Ugyanakkor az R nem ellensége, hanem partnere ezeknek: a számításigényes háttérműveletek és a kritikus csomagok belső rutinjai mögött a háttérben ma is gyakran C, C++ vagy éppen Fortran kódok futnak az optimális teljesítmény érdekében.

Forrás: Gemini

3.2 RStudio - karmester az R-hez

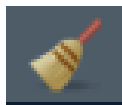
Ahogy korábban már említettük az R egy programozási nyelv, ezért az alapverzió semmilyen grafikus kezelőfelületet (*Graphical User Interface*) nem tartalmaz. A folyamatok jelentősen egyszerűsíthetők, ha a felhasználók az ingyenesen letölthető [RStudio](#) használják, amely független a programnyelvtől, de számos olyan rövidítést, egyszerűsítést (“*shortcut*”) tartalmaz, amely könnyen kezelhető környezetet biztosít a kódolás során. Az RStudio nem csupán az R-hez készített GUI, hanem magában integrál más programnyelveket is, mint például a Python, Julia, SQL, Stan stb, vagyis képes különböző nyelveken kódokat futtatni, az objektumok között pedig (feltételes) átjárhatóságot biztosít. Ezen kívül kezeli az úgynevezett *markdown* formátumot is, amely az elemzéseket reprodukálhatóvá teszi.

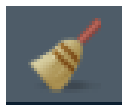
Léteznek más GUI applikációk is (pl.: [R Commander](#)), amelyek még kezelhetőbb felületet nyújtanak az R-hez, ugyanakkor ezek meglehetősen célzottak, vagyis csak bizonyos számú függvény futtatását teszik lehetővé (bár ezek kétségkívül a legnépszerűbbek), így nincs a lehetőségek meglehetősen leszűkítettek, használatukat csak akkor ajánljuk, ha a felhasználó meggyőződött róla, hogy a GUI-ban megtalálható függvények számára elegendők.

3.2.1 Az RStudio felépítése

Az RStudio szabadon felülete szabadon alakítható, így most csak az alapverzióban megtalálható részleteket mutatjuk be, de arra biztatjuk az Olvasót, hogy a *Tools -> Global Options* menüben szabadon fedezze fel a lehetőségeket.

Environment: itt található az elmentett adatok, függvények és objektumok. Az objektum nevére kattintva beletekinthetünk az adatok szerkezetébe, azonban függvény hívására nincs



lehetőség. Egy fontos dolog kiemelendő: a  ikonra kattintva minden változót kitöröl a munkaterületről. **A műveletet nem lehet visszavonni.**

Files: az aktuális mappa, amiben dolgozunk. Nem feltétlenül szükséges használni, de egyszerűsíti a munkát, ha látjuk a fájlstruktúrát, illetve könnyen tudunk hivatkozni rá. Kattintásra a fájlok nem hivatkozhatók.

Plots: minden vizualizáció itt jelenik meg, menthető el, törölhető ki, illetve tárolható a memóriában.

Packages: az R lelkét azok a csomagok adják, amiket a tudományos közösség fejlesztett. Itt a letöltött és telepített csomagokat láthatjuk. Újakat vagy a konzolból (lásd lentebb), vagy itt, az *Install* gombra kattintva telepíthetünk. A közösség világszerte működtet szervereket (Magyarországon az ELTE), amelyek a letölthető modulok folyamatosan tükrözött tárhelyei. Az itt megtalálható csomagok folyamatos fejlesztés alatt járnak, ellenőrzöttek. Előfordulhatnak olyan modulok is, amelyek nem találhatók meg a szerveren, ezeket külön tudjuk telepíteni. Az, hogy egy csomag nem található meg a szerveren nem feltétlenül jelenti azt, hogy nem megbízható, sokkal inkább csupán azt, hogy jelenleg nincs igazán “gazdája” (aki készítette nem kívánja karban tartani, aktualizálni stb.). Elsősorban a szerverről letöltött csomagok használatát javasoljuk.

Help: minden csomaghoz tartozik egy *help* kiegészítés is, ami kimerítően tartalmazza az összes függvény leírását, példáulval együtt.

Console: a konzol az RStudio és az R kommunikációt biztosítja, lényegében itt tud a felhasználó az R-rel közvetlenül kommunikálni. Minden itt, valamint a fenti munkaterületen kiadott utasítást az R a konzolban hatja végre és a válaszokat is elsősorban ott adja.

Workspace: ide írható, minden olyan utasítás, komment, szöveg, amit el szeretnénk magunknak tárolni, formázni stb.

3.3 A workspace használata, kódok írása

Az elemzés elsősorban a munkaterületen zajlik, a konzolba utasításokat közvetlenül nem szokás írni. Nagyon fontos, hogy a munkaterületet elsősorban szöveg írására alkalmazzuk. Bármilyen, amit ide írunk szöveggé lesz eltárolva, a csak úgy beírt kódok közvetlenül nem futtathatók,

ehhez külön szükséges jelezni, hogy a következő szakaszban egy kód fog következni.

A gombra kattintva illeszthető be programkód a kiválasztott nyelven. Jellemzően minden nyelvhez elérhető fordító csomag, így nem kell több nyelven írni a jegyzetet, hanem elegendő csupán az R alapú fordítót alkalmazni (SQL nyelv esetén tipikusan ez az eljárás).

Kódokat a jegyzetfüzetből a Windows operációs rendszerben a CTRL+ENTER kombináció segítségével tudjuk futtani soronként (több sor is kijelölhető), vagy a  gombra kattintva tömbönként (a kijelölt helytől lefelé vagy fölfelé minden kódot futtasson).



Az R-ben minden, amivel dolgozunk, egy **objektum**. Statisztikai szempontból ez azt jelenti, hogy a modelljeink, az adataink és a függvényeink egyenrangúak a memóriában. Fontos megérteni a különbséget a *konzol* (ahol az utasítások azonnal végrehajtnak) és a munkaterületi *script* (ahol a reprodukálható kutatás dokumentálható) között.

3.4 Csomagok, package-ek telepítése, használata

Ahogy már fentebb említettük, az R függvényeit a tudományos közösség tartja fenn. A megírt függvényeket csomagokban tölthetjük le valamelyik ún. CRAN szerverről. Fontos, hogy letöltés után be kell tölteni a csomagot, hogy használni tudjuk. Ha egyszer letöltöttünk egy csomagot, akkor később már nem szükséges újra letölteni, de minden alkalommal be kell tölteni.

A letöltés történhet konzolból, vagy GUI-ból. Mivel nincs nagy különbség a kettő között, ezért csak a konzolos változatot mutatjuk be. Telepítsük fel a **dplyr** nevű csomagot, ami az adatmanipuláció széles körben alkalmazott modujla.

A szükséges függvények a következők:

```
install.packages("dplyr") # fontos, hogy letöltéskor mindig idézőjelbe kell tenni a csomag nevet  
library(dplyr) # betöltéskor ugyanakkor már nem szükséges az idézőjel, hiszen a csomag már ott van
```

Jellemzően szoktunk kapni valamilyen kommunikációt a sikeres letöltésről, telepítésről és betöltésről, bizonyos esetben akár figyelemfelhívást is.

Warning! figyelmeztetés

Csomagok betöltésekor, valamint függvények futtatásakor sok esetben kaphatunk **Warning!** üzenetet. Ezek legtöbbször nem jelentik azt, hogy a művelet ne futott le volna sikeresen, ne lenne érvényes az eredmény, de felhívja a figyelmet, hogy valamilyen feltétel sérült, nem teljesült, vagy esetleg figyelmeztet minket, hogy milyen feltételek mellett érvényesek az eredmények. Olvassuk el, és döntsük el, hogy a figyelmeztetés hogyan érinti a mi munkánkat.

Tűzfalak

Ahhoz, hogy az R le tudja tölteni a csomagokat, ki kell lépnie a tűzfalon kívülre. Ezeket a legtöbb vállalati rendszer tiltani szokta, ezért fontos, hogy az IT támogatólag lépjen fel, és adja hozzá kivételként a szoftvert. A CRAN szerver szigorúan ellenőrzött, egy csomagot akár több millióan is letölthetnek, így a víruskockázat szinte nem létezik.

Ha nagyon gyorsan szükséges egy csomag, akkor azok minden esetben elérhető .zip fájlként (ilyenkor egy Google keresés vezet el a helyes CRAN vagy github oldalra). Ha ezt a fájlt le lehet tölteni valamilyen módon, akkor ezután közvetlenül telepíthető a Tools -> Install

Packages menüből, ahol az “Install from” lenyíló menüből kiválasztjuk a .zip opciót. Sok esetben azonban ez sem járható út, többek között azért, mert egy csomag a legtöbb esetben más csomagok függvényeire is támaszkodik, amit egy `install.packages()` függvény automatikusan letölt. Ha csak annak a csomagnak a .zip fájlját töltjük le, amit használni szeretnénk, akkor elképzelhető, hogy még sok másik csomagot le kellene töltenünk, ami szinte kivitelezhetetlen.

Ilyenkor két alternatíva jön szóba, amíg az IT segít:

- Google Colab, ami egy alapvetően ingyenes felhőalapú programozási felület és képes R kódokat is futtatni, csomagokat telepíteni. Mivel ez a Google szerverén fut, ezért semmilyen engedély nem szükséges a csomagot telepítéséhez.
- RStudio Server, amit a Posit tart fenn, és egy felhőalapú alapvetően szintén ingyenes szolgáltatás.

Mindkét megoldás alapverziója ingyenes, ám rendkívül lassú, és mivel adatainkat ilyenkor egy külső szerverre helyezzük, ezért komoly adatvédelemi aggályok is felmerülnek. Mivel azonban nem vesz el számítókapacitás a saját helyi gépünktől, ezért gyakran kényelmes megoldást jelentenek.

3.5 Mesterséges intelligencia és az R

A jelenleg használatos népszerű nagy nyelvi modellek mind ismerik az R kódnyelvet és kiváló válaszokat tudnak adni a kérdésekre. Bármelyik NLP használata erősen javallt, különösen akkor, ha hosszú, repetitív elemeket tartalmazó kódot kell írunk. Jó magyarázatokat kaphatunk és a hibákat is jellemzően jól ismerik fel, de természetesen teljesen ne bízzuk rá magunkat, különösen egy olyan területen, amit kevésbé ismerünk.

4 R alapok

Az adatok beolvasása előtt tekintsük át a legfontosabb parancsokat, adatstruktúrákat, függvényeket.

4.1 Műveletek változókkal

4.1.1 Változó definiálása

A változók definiálása elsősorban a (`<-`) operátorral történik, de bizonyos esetekben alkalmazható a (`=`) operátor is. Ez utóbbit nem javasoljuk, mert kavarodást okozhat a logikai (`==`) azonosságot jelentő operátorral.

```
x<-1 # tároljuk el az értéket  
print(x) # irassuk ki az értéket
```

```
[1] 1
```

```
x # a kiíratás így is működik
```

```
[1] 1
```

```
y<-7  
  
z<-sqrt(x+y) # hivatkozzunk egy változókra a függvényben  
z
```

```
[1] 2.828427
```

Láthattuk, hogy skalárt operátor nélkül tudunk definiálni, vektor rögzítéséhez azonban a `c()` operátor alkalmazása szükséges. Mindenképpen meg kell jegyeznünk, hogy technikai értelemben az R nem ismeri a skalárt, hanem egy egyelemű vektorként kezeli. Ennek értelmében a helyes definiálás az `x<-c(1)` lett volna, de egy elem esetén a `c()` operátor nélkül is elfogadott.

Egy vektorban számokat és szöveget is tudunk tárolni, akár vegyesen is, de ez utóbbi esetben szöveg (string) típusként azonosítjuk a változókat. Szöveg esetén nagyon fontos, hogy a szöveg idézőjelek közé teendő. Ha nem így teszünk, akkor a beírt szöveg hivatkozásként lesz értelmezve a változókészletből.

```
x<-c(1, 2, 3.467, 4.01)
y<-c("cat", "dog")
z<-c(1, "cat", TRUE)
```

Érdemes még megemlíteni a szekvenciák (`seq()`, illetve rövidebben `from:to`) és az ismétlés függvényeket (`rep()`).

```
x_grid <- 1:10           # Egész számok 1-től 10-ig [10 elem].
p_grid <- seq(from=0, to=1, by=0.05) # Valószínűségi rács.
groups <- rep(c("A", "B"), times=5)  # Ismétlődő minták generálása [10 elem].
```

4.1.2 Indexálás

A Python programozási nyelvhez képest az egyik legnagyobb különbség a vektorok indexálása. Az R-ben egy vektor első elemének az indexe **1**. Ezzel szemben a Python-ban az első elem indexe a 0. A nulladik indexet az R nem ismeri. Egy vektor elemére vagy elemeire való hivatkozás a `változó[]` módon történik.

```
x[4] # a vektor második eleme
```

```
[1] 4.01
```

```
x[1:3] # a vektor első három eleme
```

```
[1] 1.000 2.000 3.467
```

! Változó tárolása

Amennyiben nem mentjük el a változót az `<-` operátorral, úgy az R csupán végrehajtja a parancsot, de **nem írja felül a változót**. Ügyeljünk rá, hogy felülírás esetén (például `x<-x[1:3]`) az R nem kér megerősítést.

Egy kellemes funkció, hogy a negatív indexálás az elem elhagyását jelenti a vektorból.

```
x[-1] # az első elem elhagyása a vektorból
```

```
[1] 2.000 3.467 4.010
```

```
x[-c(1,3)] # az első és a harmadik elem elhagyása a vektorból
```

```
[1] 2.00 4.01
```

```
x[-c(1:3)] # az első három elem elhagyása a vektorból
```

```
[1] 4.01
```

```
p_grid[-seq(from=2, to=21, by=2)] # minden második elem elhagyása a vektorból
```

```
[1] 0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0
```

Az indexálás nagy előnye, hogy logikai operátorokat ($\leq$, $\geq$) is kezel.

```
x[x>3] # 3-nál nagyobb értékű elemek
```

```
[1] 3.467 4.010
```

! Logikai operátorok

R-ben az azonosságot nem “=” jellel, hanem dupla “==” jellel jelöljük. A negáció pedig “!=” operátortal jelölendő. A logikai ÉS a “&” operátorral, míg a VAGY a “|” (magyar billentyűzetten ALTGR+W) operátorral használatos. Minden esetben a vonatkozó változót az “&” és a “|” operátorok mindkét oldalán definiálni kell.

```
groups[groups=="A"] # csak "A" karakterek a vektorban
```

```
[1] "A" "A" "A" "A" "A"
```

```
p_grid[p_grid<=0.2 | p_grid>=0.8] # 0.2-nél kisebb vagy egyenlő VAGY 0.8-nál nagyobb vagy egyenlő
```

```
[1] 0.00 0.05 0.10 0.15 0.20 0.80 0.85 0.90 0.95 1.00
```

```
p_grid[p_grid<0.2 & p_grid>0.8] # az eredmény egy üres vektor, mert nincs olyan szám, ami ki
```

```
numeric(0)
```

4.1.3 Vektorizáció

Az R-ben a vektorokkal végzett aritmetikai műveletek az elemek mentén hajtódnak végre. A “*” operátorral végzett szorzat tehát nem skaláris. Fontos, hogy csak azonos hosszúságú vektorokon végezhetők el a műveletek.

```
x <- c(1, 2, 3)
y <- c(10, 20, 30)
x * y # nem skaláris szorzat
```

```
[1] 10 40 90
```

A skaláris szorzat elvégzéséhez a “%*%” operátor használata szükséges.

```
x%*%y # skaláris szorzat
```

```
      [,1]
[1,] 140
```

! Újrafelhasználási szabály

Rendkívül fontos felhívni a figyelmet, hogy az R képes eltérő hosszúságú vektorokon is műveletek végrehajtani. Ilyenkor a rövidebb vektort ciklikusan ismétli (újrafelhasználja), amíg el nem éri a hosszabb vektor méretét. Ez a tulajdonság rendkívül hasznos például expozícióval való súlyozáskor, de veszélyes hibaforrás is lehet, ha a hosszak nem egymás többszörösei.

4.2 Hiányzó adatok és kezelésük

R-ben a hiányzó adatokat az NA operátor jelöli. Ha egy vektor hiányzó elemet tartalmaz, akkor az elemmel végrehajtott művelet automatikusan NA értéket fog kapni. Amennyiben skaláris műveletet hajtunk végre hiányzó elem mellett, akkor az eredmény vagy NA érték lesz, vagy hibaüzenet. Ez az R egy kellemes tulajdonsága, hogy minden esetben felhívja a figyelmet arra, hogy vannak hiányzó adatok.


```
x<-c(10, 20, NA, 30, 40)
y<-c(5, 15, 25, 35, 45)
x+y
```

```
[1] 15 35 NA 65 85
```

```
x*y
```

```
[1] 50 300 NA 1050 1800
```

```
x/y
```

```
[1] 2.0000000 1.3333333 NA 0.8571429 0.8888889
```

```
x%%y
```

```
      [,1]
[1,]    NA
```

4.3 Alapstatisztikák

Az R igazi ereje a statisztikai számítások esetén mutatkozik meg. A leggyakrabban használt statisztikai függvények megtalálhatók az alapverzióban, nem szükséges semmilyen csomag telepítése, betöltése. Többet között ilyen a `mean`, `sd` vagy a `summary` függvények is. Az R alapverziójában megtalálható függvények száma ezer feletti. Néhány példát áttekintünk, amelyek a leíró statisztikához szükségesek.

```
x<-rnorm(1000, mean = 0, sd = 1) # egy standard normális eloszlású 1000 elemű véletlen vektor
```

Alapértelmezett értékek

A legtöbb függvény különböző hiperparaméterek mellett definiálható. A legtöbb paraméternek van alapértelmezett értéke vagy függvénye, így ha az számunkra megfelelő, akkor nem kell külön definiálni. Az `rnorm` függvény például három paraméter mellett definiálható: az elemszám (`n`) az átlag (`mean`) és a szórás (`sd`), a függvénye pedig `rnorm(n, mean, sd)`. Ezek az alapértelmezett értékek $\mu = 0$ és $\sigma = 1$ (a standard normális eloszlás értékei), így ha ezek megfelelnek számunkra, akkor csupán az elemszámot kell megadni,

vagyis futtatható az `rnorm(n)` függvény a paraméterek nélkül. Ha el kívánunk térni az alapértelmezett paraméterektől, akkor a módosított értéket definiálni szükséges. Az alapértelmezett értékekről mindig információt nyújt a függvény *help* melléklete, amit a `?függvény` paranccsal kérhetünk le.

Most végezzünk el néhány alapvető statisztikai becslést az előbb definiált vektorral.

```
mean(x) # várható érték
```

```
[1] 0.003881507
```

```
sd(x) # szórás
```

```
[1] 0.9969929
```

```
summary(x) # leíró statisztika
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
-3.551000	-0.629262	0.016105	0.003882	0.682330	3.110908

! Véletlen változók

Ideális esetben a véletlen változó valóban véletlen módon generálódik a processzor órajele alapján. Egy szerverre kötött gépek esetén a véletlen szám a szerver processzorának órajele alapján generálódik, így a szerver minden gépen várhatóan azonos lesz.

Fontos azonban, hogy minden futtatás új véletlen számot generál. Ha meg kívánjuk őrizni a véletlen számot és más gépeken is szeretnénk ugyanazt kapni, úgy a `set.seed()` paranccsal és egy megadott számmal ez megvalósítható. [Douglas Adams](#) után szabad a közösség gyakran a `set.seed(42)` formátumot szokta alkalmazni.

4.4 Faktorok

A biztosításmatematikai modellekben a prediktorok jelentős része kategória típusú (pl. régió, bónusz-malusz osztály). R-ben ezeket faktorként kezeljük, ami belsőleg egész számokból és a hozzájuk tartozó szintekből (levels) áll. Fontos megjegyezni, hogy a legtöbb modellben a faktorok automatikusan dummy változókká alakulnak a modellmátrix felépítésekor.

```

regio_k <- c("Pest", "Vidéki város", "Pest", "Község", "Vidéki város")
regio_faktor <- factor(regio_k) # az R automatikusan ábécé sorrendbe teszi a szinteket

# Gyakran szükség van egy bázisszint (referencia) rögzítésére a modellezéshez.

# ezt a levels argumentummal kontrollálhatjuk
regio_faktor <- factor(regio_k, levels = c("Pest", "Vidéki város", "Község"))

# Ellenőrizzük a szinteket - a modellezésnél az első lesz a bázisszint.
levels(regio_faktor)

```

```
[1] "Pest"          "Vidéki város" "Község"
```

4.5 Adattáblák (data frames)

Az adattábla az R legfontosabb objektumtípusa. Egy lista, ahol minden elem azonos hosszúságú. Megfelel egy SQL táblának vagy Excel munkalapnak, de az oszlopok szigorúan típusosak. Nem keverendő a mátrix formátummal, ami kizárólag azonos típusú adatot tud tárolni. A legtöbb esetben az adatimportálás során eleve adattáblába történik a beolvasás. Könnyen, változónévvel vagy indexszel is hivatkozható tömbökről van szó.

Ha mi magunk hozzuk létre a táblát, akkor külön definiálandó minden oszlop (változó).

```

df <- data.frame(
  id = 1:5,
  kor = c(25, 45, 30, 55, 40),
  kar_esemeny = c(FALSE, TRUE, FALSE, FALSE, TRUE)
)

```

! Logikai értékek definálása

Az igaz-hamis értékek eleve faktorváltozóként kerülnek tárolásra. Nagyon fontos a definálás során, hogy **mindig nagybetűvel** kell őket írni, vagy is **TRUE** vagy **FALSE**.

A data frame objektum már kellemes módon, külön ablakban jeleníthető meg, ha az *environment* ablakban az objektum nevére kattintunk.

Az adattáblák legnagyobb előnye, hogy könnyen hivatkozhatók az `\$` operátorral a változó neve után írva.

```
df$kor
```

```
[1] 25 45 30 55 40
```

Természetesen indexálással is működik, mivel azonban már nem vektorral, hanem táblával dolgozunk, ezért két indexet kell megadnunk, egy sort és egy oszlopot, ebben a sorrendben. Generikusan `variable[,]` módon írhatjuk fel a hivatkozást. Ha mindkét paramétert megadjuk, akkor értelemszerűen egy elemre, a metszéspontra hivatkozunk.

```
df[1,2] # első sor, második oszlop metszete, azaz az első elem
```

```
[1] 25
```

```
# Ha elhagyjuk valamelyik paramétert, akkor a teljes sorra/oszlopra hivatkozunk
df[,3] # minden elem (minden sor) a harmadik oszlopból
```

```
[1] FALSE TRUE FALSE FALSE TRUE
```

```
df[3,] # harmadik megfigyelés (harmadik sor) minden eleme (minden oszlopa)
```

```
  id kor kar_esemeny
3  3  30         FALSE
```

```
df[2:4,] # másodiktól a negyedik sorig minden oszlop
```

```
  id kor kar_esemeny
2  2  45         TRUE
3  3  30         FALSE
4  4  55         FALSE
```

```
df[2:4,1:2] # a másodiktól a negyedik sorig, az első két oszlop
```

```
  id kor
2  2  45
3  3  30
4  4  55
```

Természetesen működik a kettő kombinációja is, de csak akkor ha egy változót indexálunk. Ilyenkor már nem adattáblánk, hanem vektorunk lesz (hiszen csak egy változóra hivatkozunk a táblából).

```
df$kor[2:3] # a kor változó második és harmadik eleme
```

```
[1] 45 30
```

5 Adatok importálása és adatmanipuláció

5.1 Adatok importálása

Az R a legtöbb adattípust tudja be tudja olvasni. Azonban saját adattípussal rendelkezik **.RData** formátumban, amennyiben R-ben dolgozunk úgy, érdemes ezt a típust alkalmazni a saját fájlok esetén és nem visszakonvertálni egy másik formátumra.

Beépített adatimportáló funkciók léteznek, ezeket érdemes használni. Mindazonáltal érdemes az adattisztítást előre elvégezni egy nagyobb rugalmasságot engedő GUI-val rendelkező szoftverben (pl: Excel), ha az úgy gyorsabb.

A betöltés a File -> Import Dataset -> From Text párbeszéd ablakon keresztül történik. Mind a “base”, mint a “readr” megoldás alkalmazható, ez utóbbi talán kényelmesebb.

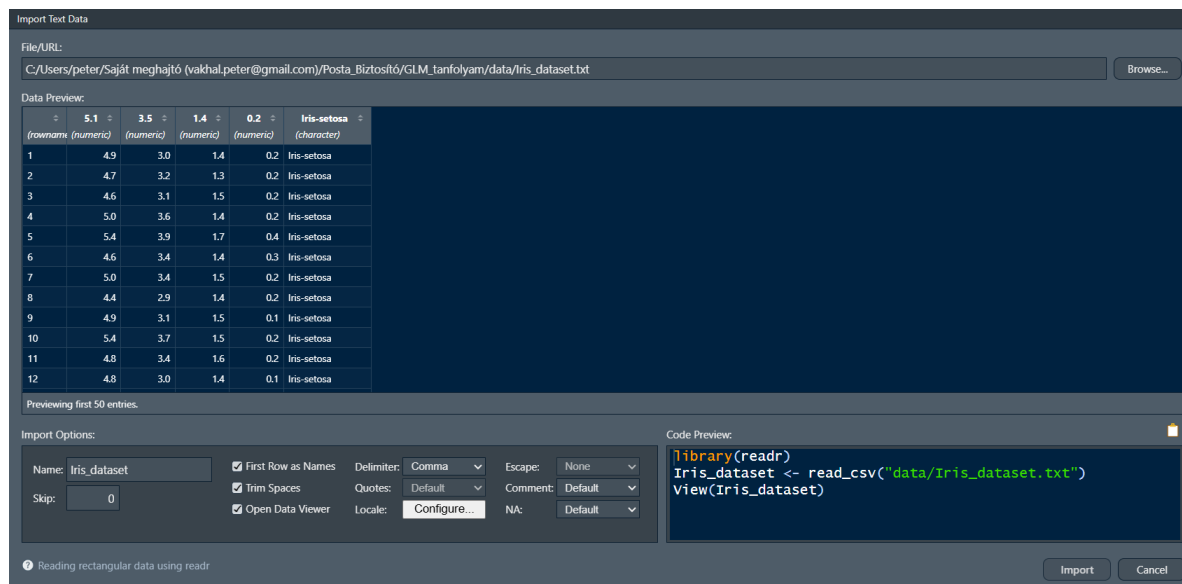


Figure 5.1: readr

A párbeszédablak magától értetődő. A beolvasás során a csomag megpróbálja felismerni az adattípust, amennyiben nem jól tette, úgy változtathatunk a változó nevére kattintva. Ügyeljünk rá, hogy gyakran előfordul, hogy külön oszlopot hoz létre a megfigyelés sorszámának.

Amennyiben erre nincs szükségünk, úgy ugorjuk át ezt az oszlopot, azaz állítsuk *skip* értékre. A jobb alsó sarokban láthatjuk a generált kódot, amit akár ki is másolhatunk, de az import gombra kattintva is végrehajtódik a függvény.

```
library(readr) # töltsük be a csomagot (előre van telepítve)
iris <- read_csv("data/Iris_dataset.txt") # olvassuk be az adatokat
```

```
Rows: 150 Columns: 5
```

```
-- Column specification -----
```

```
Delimiter: ","
```

```
chr (1): Species
```

```
dbl (4): Sepal.Length, Sepal.Width, Petal.Length, Petal.Width
```

```
i Use `spec()` to retrieve the full column specification for this data.
```

```
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Ugyanezen az elvan működik az .xlsx (File -> Import Dataset -> From Excel) vagy a .csv (itt ugyanúgy a fentebb ismertetett From Text menüpontra van szükség) fájlok betöltése.

5.2 Adatmanipuláció a dplyr csomag segítségével

A dplyr csomag a legszélesebb körben alkalmazott adatmanipulációs csomag, amelynek szinte végtelen kombinációs lehetőségei vannak. Most a leggyakoribb műveleteket tekintjük át, de arra ösztönözzük az Olvasót, hogy szükség esetén konzultájon valamelyik NLP alapú ügynökkel, hogy bonyolult manipulációkat is végre tudjon hajtani.

A dplyr csomag egyik legnagyszerűbb eleme a *pipeline* operátor, amivel objektumokat tudunk összekapcsolni. Az R ugyanis alapesetben csak a függvényen belül tudja kezelni az objektumokat, ez sokszor kényelmetlen, ha egymásba szeretnék ágyazni több függvényt is. A *pipeline* operátor a `%>%` módon definálható.

Nagyon fontos, hogy a dplyr csomag csak a data frame alapú objektumokat tudja kezelni!

5.2.1 select()

Ez a művelet a változók (oszlopok) egy részhalmazának kiválasztását végzi el. A változók nevét nem kell idézőjelbe tenni.

```
library(dplyr) # töltsük be a csomagot
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
select(iris, Sepal.Length, Sepal.Width) # a szírom hosszúság és szélesség kiválasztása
```

```
# A tibble: 150 x 2
  Sepal.Length Sepal.Width
      <dbl>         <dbl>
1         5.1         3.5
2         4.9         3
3         4.7         3.2
4         4.6         3.1
5          5          3.6
6         5.4         3.9
7         4.6         3.4
8          5          3.4
9         4.4         2.9
10        4.9         3.1
# i 140 more rows
```

```
# Ugyanez %>% operátorral
iris %>%
  select(Sepal.Length, Sepal.Width)
```

```
# A tibble: 150 x 2
  Sepal.Length Sepal.Width
      <dbl>         <dbl>
1         5.1         3.5
2         4.9         3
```



```

3          4.7          3.2
4          4.6          3.1
5          5           3.6
6          5.4          3.9
7          4.6          3.4
8          5           3.4
9          4.4          2.9
10         4.9          3.1
# i 140 more rows

```

```
select(iris, -Species, -Petal.Length) # változók kihagyása
```

```

# A tibble: 150 x 3
  Sepal.Length Sepal.Width Petal.Width
      <dbl>         <dbl>         <dbl>
1         5.1         3.5         0.2
2         4.9         3          0.2
3         4.7         3.2         0.2
4         4.6         3.1         0.2
5         5          3.6         0.2
6         5.4         3.9         0.4
7         4.6         3.4         0.3
8         5          3.4         0.2
9         4.4         2.9         0.2
10        4.9         3.1         0.1
# i 140 more rows

```

```
select(iris, contains("Petal")) # a "Petal" karakterláncot tartalmazó változónevek kiválasztása
```

```

# A tibble: 150 x 2
  Petal.Length Petal.Width
      <dbl>         <dbl>
1         1.4         0.2
2         1.4         0.2
3         1.3         0.2
4         1.5         0.2
5         1.4         0.2
6         1.7         0.4
7         1.4         0.3
8         1.5         0.2
9         1.4         0.2

```

```
10          1.5          0.1
# i 140 more rows
```

A `contains()` függvény helyett értelemszerűen alkalmazhatók a `'starts_with'`, `'ends_with'` és a `matches()` függvények is.

5.2.2 filter()

Megfigyelések (sorok) kiválasztásához egy adott kritérium alapján használjuk a `filter()` függvényt.

```
filter(iris, Species == "setosa") # csak a "setosa" fajta kiválasztás
```

```
# A tibble: 50 x 5
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
      <dbl>       <dbl>       <dbl>       <dbl> <chr>
1         5.1         3.5         1.4         0.2 setosa
2         4.9         3         1.4         0.2 setosa
3         4.7         3.2         1.3         0.2 setosa
4         4.6         3.1         1.5         0.2 setosa
5          5          3.6         1.4         0.2 setosa
6         5.4         3.9         1.7         0.4 setosa
7         4.6         3.4         1.4         0.3 setosa
8          5          3.4         1.5         0.2 setosa
9         4.4         2.9         1.4         0.2 setosa
10        4.9         3.1         1.5         0.1 setosa
# i 40 more rows
```

```
# Ugyanez %>% operátorral
iris %>%
  filter(Species=="setosa")
```

```
# A tibble: 50 x 5
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
      <dbl>       <dbl>       <dbl>       <dbl> <chr>
1         5.1         3.5         1.4         0.2 setosa
2         4.9         3         1.4         0.2 setosa
3         4.7         3.2         1.3         0.2 setosa
4         4.6         3.1         1.5         0.2 setosa
5          5          3.6         1.4         0.2 setosa
```

```

6          5.4          3.9          1.7          0.4 setosa
7          4.6          3.4          1.4          0.3 setosa
8          5          3.4          1.5          0.2 setosa
9          4.4          2.9          1.4          0.2 setosa
10         4.9          3.1          1.5          0.1 setosa
# i 40 more rows

```

```

filter(iris, Species == "setosa" | Species == "virginica") # "setosa" vagy "virginica" típus

```

```

# A tibble: 100 x 5

```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
	<dbl>	<dbl>	<dbl>	<dbl>	<chr>
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
7	4.6	3.4	1.4	0.3	setosa
8	5	3.4	1.5	0.2	setosa
9	4.4	2.9	1.4	0.2	setosa
10	4.9	3.1	1.5	0.1	setosa

```

# i 90 more rows

```

```

filter(iris, Species == "setosa" & Sepal.Length < 5) # 5mm-nél kisebb szirmú "setosa" típus

```

```

# A tibble: 20 x 5

```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
	<dbl>	<dbl>	<dbl>	<dbl>	<chr>
1	4.9	3	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	4.6	3.4	1.4	0.3	setosa
5	4.4	2.9	1.4	0.2	setosa
6	4.9	3.1	1.5	0.1	setosa
7	4.8	3.4	1.6	0.2	setosa
8	4.8	3	1.4	0.1	setosa
9	4.3	3	1.1	0.1	setosa
10	4.6	3.6	1	0.2	setosa
11	4.8	3.4	1.9	0.2	setosa
12	4.7	3.2	1.6	0.2	setosa

13	4.8	3.1	1.6	0.2	setosa
14	4.9	3.1	1.5	0.1	setosa
15	4.9	3.1	1.5	0.1	setosa
16	4.4	3	1.3	0.2	setosa
17	4.5	2.3	1.3	0.3	setosa
18	4.4	3.2	1.3	0.2	setosa
19	4.8	3	1.4	0.3	setosa
20	4.6	3.2	1.4	0.2	setosa

Több függvényt a `%>%` könnyen egymásba ágyazhatunk.

```
iris %>%
  filter(Sepal.Length < 5) %>% # 5mm-nél kisebb szirmok
  select(Sepal.Width, Sepal.Length, Species) %>% # csak ezek a változók
  head(4) # csak az első négy megfigyelés kiírása
```

```
# A tibble: 4 x 3
  Sepal.Width Sepal.Length Species
      <dbl>      <dbl> <chr>
1         3         4.9 setosa
2        3.2         4.7 setosa
3        3.1         4.6 setosa
4        3.4         4.6 setosa
```

5.2.3 mutate()

A `mutate()` függvénnyel új változót adhatunk az adattáblánkhoz. Ez gyakran elő fog fordulni adatmanipuláció során, mert sok esetben segédváltozóra van szükség egy-egy bonyolultabb kimutatás előállításához. Ellátsuk elő például az értékek arányait.

```
iris %>%
  mutate(Petal.Shape = Petal.Width / Petal.Length,
         Sepal.Shape = Sepal.Width / Sepal.Length) %>%
  select(Species, Petal.Shape, Sepal.Shape)
```

```
# A tibble: 150 x 3
  Species Petal.Shape Sepal.Shape
    <chr>      <dbl>      <dbl>
1 setosa    0.143      0.686
2 setosa    0.143      0.612
```

```

3 setosa      0.154      0.681
4 setosa      0.133      0.674
5 setosa      0.143      0.72
6 setosa      0.235      0.722
7 setosa      0.214      0.739
8 setosa      0.133      0.68
9 setosa      0.143      0.659
10 setosa     0.0667     0.633
# i 140 more rows

```

5.2.4 group_by és summarize

Számos adatelemzési feladat megközelíthető a split-apply-combine paradigma segítségével:

- bontsuk fel az adatokat csoportokra,
- alkalmazzunk valamilyen elemzést mindegyik csoportra,
- Kombináljuk az eredményeket.

A dplyr ezt nagyon egyszerűvé teszi a group_by és summarize függvények használatával. A group_by minden csoportot egyetlen soros összefoglalóvá csuk össze. Argumentumként azokat az oszlopneveket veszi, amelyek azokat a **kategorikus (faktor)** változókat tartalmazzák, amelyekre az összefoglaló statisztikákat ki szeretnénk számítani.

```

iris %>%
  group_by(Species) %>%
  summarise(Mean.Petal.Length = mean(Petal.Length)) # átlagos hossz kiszámítása fajonként

```

```

# A tibble: 3 x 2
  Species      Mean.Petal.Length
  <chr>          <dbl>
1 setosa          1.46
2 versicolor      4.26
3 virginica        5.55

```

Több összefoglaló statisztikát is számíthatunk.

```

iris %>%
  mutate(Petal.Long = Petal.Length > 5) %>% # új változó, ami logikai érték (egy "=" jel),
  group_by(Species, Petal.Long) %>% # fajok és az előbb létrehozott logikai változó alapján
  summarise(Mean.Petal.Length = mean(Petal.Length), # csoportátlag
            n.Petals = length(Petal.Length), # elemszám

```

```
sd.Petal.Length = sd(Petal.Length), # szórás
SE.Petal.Length = sd(Petal.Length) / sqrt(length(Petal.Length))) # relatív hiba
```

`summarise()` has grouped output by 'Species'. You can override using the `.groups` argument.

```
# A tibble: 5 x 6
# Groups:   Species [3]
  Species Petal.Long Mean.Petal.Length n.Petals sd.Petal.Length SE.Petal.Length
  <chr>    <lgl>          <dbl>    <int>    <dbl>          <dbl>
1 setosa  FALSE           1.46      50      0.174          0.0245
2 versico~ FALSE           4.24      49      0.459          0.0655
3 versico~ TRUE            5.1       1      NA            NA
4 virgini~ FALSE           4.87       9      0.158          0.0527
5 virgini~ TRUE           5.70      41      0.489          0.0764
```

5.2.5 arrange()

Gyakran előfordul, hogy a sorokat rendezni szeretnénk csökkenő vagy növekvő sorrendben. A dplyr segítségével ez elegánsan elvégezhető, de természetesen alkalmazható a `sort()` függvény is, ami az alapcsomag része.

```
iris %>%
  group_by(Species) %>%
  summarise(Mean.Petal.Length = mean(Petal.Length),
            n.Petals = length(Petal.Length),
            sd.Petal.Length = sd(Petal.Length),
            SE.Petal.Length = sd(Petal.Length) / sqrt(length(Petal.Length))) %>%
  arrange(-Mean.Petal.Length) # csökkenő sorrend a negatív jel miatt
```

```
# A tibble: 3 x 5
  Species Mean.Petal.Length n.Petals sd.Petal.Length SE.Petal.Length
  <chr>          <dbl>    <int>    <dbl>          <dbl>
1 virginica      5.55      50      0.552          0.0780
2 versicolor     4.26      50      0.470          0.0665
3 setosa         1.46      50      0.174          0.0245
```

5.2.6 xtabs(), table() és egyéb keresztábrás megoldások

A keresztábra készítés meglehetősen bonyolult művelet minden szoftverben, különösen, ha előtte még szükség van valamilyen adatmanipulációra. Egy keresztábrában jellemzően két faktorváltozó metszetében a gyakoriság jelenik meg. Amennyiben megvan a két faktorváltozónk úgy nem bonyolult a dolog, ha nincs, akkor elő kell állítani.

Az `xtabs()` és `table()` két előre beépített függvény, amelyek azért jók, mert későbbiekben a statisztikai függvényeket ezeket jellemzően el szokták fogadni inputként. Fontos azonban, hogy ez a kettő elemi függvény, ezért `%>%` operátorral nem mindig tudnak kapcsolódni. Legtöbbször csupán a `data=.` paraméterrel jelezni kell, hogy korábban már megadtuk az adatok forrását.

```
iris %>%  
  mutate(Petal.Long = Petal.Length > 5) %>%  
  xtabs(~ Species + Petal.Long, data=.)
```

	Petal.Long	
Species	FALSE	TRUE
setosa	50	0
versicolor	49	1
virginica	9	41

A `table()` az egyik legelemibb belső függvény, sajnos gyakran gyorsabban eredményhez jutunk, ha külön változóként elmentjük a módosított adattáblát, majd erre hivatkozva kérjük le a keresztábrát.

```
iris2 <- iris %>%  
  mutate(Petal.Long = Petal.Length > 5)  
  
table(iris2$Species, iris2$Petal.Long)
```

	FALSE	TRUE
setosa	50	0
versicolor	49	1
virginica	9	41

6 Feltáró adatelemzés és adatvizualizáció

7 Miről lesz szó a fejezetben?

Ez a fejezet az R alapvető statisztikai eszköztárának bemutatásáról szól, amit feltáró adatelemzésként (*“Explorative Data Analysis”*) ismerünk. Mivel a mutatók a standard egyetemi statisztika oktatás részét képezik, ezért külön módszertani magyarázatot nem fűzünk az indikátorokhoz. A fejezet másik részében megismertetjük az Olvasót az adatvizualizációs lehetőségekkel a `ggplot2` nevű modul felhasználásával.

8 Bevezetés a következtetéselméleti vizsgálatokba

A következtetéselmélet célja, hogy a mintából olyan következtetéseket vonjunk le, amely segítségével a populációra vonatkozó állításokat ellenőrizhetünk. Ezeket az állításokat **hipotézisként** kell megfogalmazni, ideális esetben még a vizsgálat előtt. Univariáns esetben ez a hipotézis jellemzően a populáció egy attribútumának momentumára vonatkozik, legjellemzőbben a centrális tendenciára, vagy más néven a **várható értékre**. Ebben az esetben természetesen feltételeznünk szükséges, hogy a vizsgált változónak létezik eloszlása, még ha nem is tudjuk pontosan, csupán feltételezzük ennek az eloszlásnak a típusát. Multivariáns esetben a megfogalmazott hipotézis már a változók közötti asszociációra (például korreláció, regresszió) vonatkozik.

A megfogalmazható hipotézis alapvetően a változó típusától függ:

- Folytonos változók esetén: a momentumokra, valamint a feltételezett értéktől való eltérésre.
- Diszkrét változók esetén: amennyiben a távolság értelmezhető a megfigyelések között (például Poisson-eloszlásból származó adatok esetén), úgy a fentebb leírtak becsülhetők.
- Nominális (faktor) változók esetén: a távolság ekkor nem értelmezhető, csupán gyakoriság számolható, hipotézisvizsgálat legalább kétváltozós kiterjesztés szükséges.

8.1 Hipotézisek folytonos változók esetén (univariáns)

A hipotézis a populáció valós eloszlására vagy annak egy paraméterére vonatkozó kijelentés. Vagyis léteznek olyan $x_i \in P_0 \subseteq p$ megfigyelések, amelyekre a kijelentés igaz. A hipotézisvizsgálat ennek ellenőrzését jelenti. A következőkben ezeket tekintjük át.

Mindenekelőtt azonban töltsük be az adatbázisunkat. Az adatok forrás a [Kaggle](#) adatbázisa. A változók többsége magától értetődő, csupán két változó értelmezése definiálandó:

- DUIS (“*Driving Under Influence*”): történt-e a múltban alkohol vagy más bódító hatású készítmény által befolyásolt vezetés?
- Outcome: Volt-e benyújtott kárigénye az ügyfélnek?

```
library(readr)
car_insur <- read_csv("data/Car_Insurance_Claim.csv")
```

Rows: 10000 Columns: 19

-- Column specification -----

Delimiter: ","

chr (8): AGE, GENDER, RACE, DRIVING_EXPERIENCE, EDUCATION, INCOME, VEHICLE_...

dbl (11): ID, CREDIT_SCORE, VEHICLE_OWNERSHIP, MARRIED, CHILDREN, POSTAL_COD...

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

8.1.1 Adattípusok leíró statisztikája

Az adatok típusáról az `str()` függvény nyújt bővebb információt, de ugyanezekhez az adatokhoz juthatunk, ha változólistában lenyitjuk a változóhoz tartozó menüt.

```
str(car_insur)
```

spc_tbl_ [10,000 x 19] (S3: spec_tbl_df/tbl_df/tbl/data.frame)

\$ ID : num [1:10000] 569520 750365 199901 478866 731664 ...

\$ AGE : chr [1:10000] "65+" "16-25" "16-25" "16-25" ...

\$ GENDER : chr [1:10000] "female" "male" "female" "male" ...

\$ RACE : chr [1:10000] "majority" "majority" "majority" "majority" ...

\$ DRIVING_EXPERIENCE : chr [1:10000] "0-9y" "0-9y" "0-9y" "0-9y" ...

\$ EDUCATION : chr [1:10000] "high school" "none" "high school" "university" ...

\$ INCOME : chr [1:10000] "upper class" "poverty" "working class" "working class"

\$ CREDIT_SCORE : num [1:10000] 0.629 0.358 0.493 0.206 0.388 ...

\$ VEHICLE_OWNERSHIP : num [1:10000] 1 0 1 1 1 1 0 0 0 1 ...

\$ VEHICLE_YEAR : chr [1:10000] "after 2015" "before 2015" "before 2015" "before 2015"

\$ MARRIED : num [1:10000] 0 0 0 0 0 0 1 0 1 0 ...

\$ CHILDREN : num [1:10000] 1 0 0 1 0 1 1 1 0 1 ...

\$ POSTAL_CODE : num [1:10000] 10238 10238 10238 32765 32765 ...

\$ ANNUAL_MILEAGE : num [1:10000] 12000 16000 11000 11000 12000 13000 13000 14000 13000

\$ VEHICLE_TYPE : chr [1:10000] "sedan" "sedan" "sedan" "sedan" ...

\$ SPEEDING_VIOLATIONS : num [1:10000] 0 0 0 0 2 3 7 0 0 0 ...

\$ DUIS : num [1:10000] 0 0 0 0 0 0 0 0 0 0 ...

\$ PAST_ACCIDENTS : num [1:10000] 0 0 0 0 1 3 3 0 0 0 ...

\$ OUTCOME : num [1:10000] 0 1 0 0 1 0 0 1 0 1 ...

- attr(*, "spec")=

```

.. cols(
..   ID = col_double(),
..   AGE = col_character(),
..   GENDER = col_character(),
..   RACE = col_character(),
..   DRIVING_EXPERIENCE = col_character(),
..   EDUCATION = col_character(),
..   INCOME = col_character(),
..   CREDIT_SCORE = col_double(),
..   VEHICLE_OWNERSHIP = col_double(),
..   VEHICLE_YEAR = col_character(),
..   MARRIED = col_double(),
..   CHILDREN = col_double(),
..   POSTAL_CODE = col_double(),
..   ANNUAL_MILEAGE = col_double(),
..   VEHICLE_TYPE = col_character(),
..   SPEEDING_VIOLATIONS = col_double(),
..   DUIS = col_double(),
..   PAST_ACCIDENTS = col_double(),
..   OUTCOME = col_double()
.. )
- attr(*, "problems")=<externalptr>

```

Láthatjuk, hogy numerikus és string (chr) típusú adataink vannak. Ez utóbbiakat faktorokként tudjuk kezelni. Ezekről az adatokról szeretnénk egy átfogó képet kapni. A numerikus adatokra vonatkozóan a momentumokra tudunk becsléseket kérni, a kategórikus (faktor) adatokra nézve pedig gyakoriságokat.

```
summary(car_insur)
```

ID	AGE	GENDER	RACE
Min. : 101	Length:10000	Length:10000	Length:10000
1st Qu.:249639	Class :character	Class :character	Class :character
Median :501777	Mode :character	Mode :character	Mode :character
Mean :500522			
3rd Qu.:753975			
Max. :999976			

DRIVING_EXPERIENCE	EDUCATION	INCOME	CREDIT_SCORE
Length:10000	Length:10000	Length:10000	Min. :0.0534
Class :character	Class :character	Class :character	1st Qu.:0.4172
Mode :character	Mode :character	Mode :character	Median :0.5250

			Mean :0.5158
			3rd Qu.:0.6183
			Max. :0.9608
			NA's :982
VEHICLE_OWNERSHIP	VEHICLE_YEAR	MARRIED	CHILDREN
Min. :0.000	Length:10000	Min. :0.0000	Min. :0.0000
1st Qu.:0.000	Class :character	1st Qu.:0.0000	1st Qu.:0.0000
Median :1.000	Mode :character	Median :0.0000	Median :1.0000
Mean :0.697		Mean :0.4982	Mean :0.6888
3rd Qu.:1.000		3rd Qu.:1.0000	3rd Qu.:1.0000
Max. :1.000		Max. :1.0000	Max. :1.0000
POSTAL_CODE	ANNUAL_MILEAGE	VEHICLE_TYPE	SPEEDING_VIOLATIONS
Min. :10238	Min. : 2000	Length:10000	Min. : 0.000
1st Qu.:10238	1st Qu.:10000	Class :character	1st Qu.: 0.000
Median :10238	Median :12000	Mode :character	Median : 0.000
Mean :19865	Mean :11697		Mean : 1.483
3rd Qu.:32765	3rd Qu.:14000		3rd Qu.: 2.000
Max. :92101	Max. :22000		Max. :22.000
	NA's :957		
DUIS	PAST_ACCIDENTS	OUTCOME	
Min. :0.0000	Min. : 0.000	Min. :0.0000	
1st Qu.:0.0000	1st Qu.: 0.000	1st Qu.:0.0000	
Median :0.0000	Median : 0.000	Median :0.0000	
Mean :0.2392	Mean : 1.056	Mean :0.3133	
3rd Qu.:0.0000	3rd Qu.: 2.000	3rd Qu.:1.0000	
Max. :6.0000	Max. :15.000	Max. :1.0000	

⚠ Minden diszkrét adat kategorikus?

Sok adatbázisban előfordulnak olyan diszkrét értékek, amelyekről el kell döntenünk, hogy kategorikus vagy numerikus adatról van-e szó, és ennek megfelelő adattípust kell hozzárendelni az adatokhoz. Ha ezt elmulasztjuk, akkor fennáll annak a kockázata, hogy egy modellben nem értelmezhető eredményt fogunk kapni.

A kategorikus adatok kiválasztásánál elsősorban ezt a kérdést kell feltenni: *Értelmezhető-e távolság az adatok között?* Itt sokkal inkább szakmai szempontok bújnak meg, mintsem matematikai-statisztikai jellegűek. Amennyiben a távolság nem értelmezhető, úgy az átlag is semmitmondó.

Adatainkban például numerikus adatként van tárolva a gyorsajtások (*“speeding violations”*) száma. A többség nem követett el semmilyen vétséget (ezt a medián értékből lehet sejteni), míg vannak notórius gyorsajtók, akik egynél jóval több esetet könyvelhettek el

maguknak. A különbség két érték között értelmezhető, a két eset eggyel több, mint az egy és kettővel, mint a nulla. Fontos azonban megjegyezni valamit, amit az elemző csak “érez”, de matematikai szempontból “nem látszik”. Mivel a távolság értelmezhető, tudjuk, hogy akinek csak 22 gyorsajtása van (ez a maximum), az éppen 22-szer “rosszabb”, mint akinek csak egy, de csak kétszer olyan “rossz”, mint akinek 11 van. Valóban “kétszeres” a különbség két vezető között 11 és 22 gyorsajtással? Vagy igazából 4-5 gyorsajtás után már mindegy? Talán állandóan gyorsan hajt a 11-szeres és a 22-szeres is, ez utóbbit csupán többször kapták el. Érdekes lehet ebben az esetben egy új változót rögzíteni az ügyfélhez, akár egy binárisat is lehet: *notórius gyorsajtó*? Ehhez nem kényszerítjük a modellünk később, hogy különbséget tegyen két ügyfél között és mondjuk kevesebb díjat tarifáljon a 11-szeres gyorsajtónak a 22-szer gyorsajtóhoz képest.

Most vizsgáljuk meg a tulajdonosi viszonyt megtestesítő (“*vehicle ownership*”) változót, amely numerikusként van kódolva, de láthatóan csak két értéket [0;1] vesz fel. Értelem-szerűen ez a változó nem numerikus, hanem kategorikus, ezért majd át kell kódolni faktorváltozóra.

8.1.2 Középértékek

Centrális mutatók kizárólag folytonos változók esetén számíthatók, a szintaxis meglehetősen egyszerű. Használjuk a *credit score* változót a feladatra.

```
mean(car_insur$CREDIT_SCORE) # minden bizonytal tartalmaz hiányzó adatot a vektor
```

```
[1] NA
```

```
table(is.na(car_insur)) # valóban vannak hiányzó adataink
```

```
FALSE  TRUE
188061  1939
```

```
mean(car_insur$CREDIT_SCORE, na.rm=TRUE) # hagyjuk figyelmen kívül a hiányzó adatokat
```

```
[1] 0.5158128
```

Középérték mutató még a medián, nem paraméteres próbák esetén sokszor inkább ez használatos.

```
median(car_insur$CREDIT_SCORE, na.rm=TRUE)
```

```
[1] 0.5250328
```

8.1.3 Diszperzitási mutatók

A szórás és a variancia számítása szintén rendkívül egyszerű, azonban a hiányzó adatokra itt is korrigálni szükséges.

```
sd(car_insur$CREDIT_SCORE, na.rm=TRUE) # szórás
```

```
[1] 0.1376881
```

```
var(car_insur$CREDIT_SCORE, na.rm=TRUE)
```

```
[1] 0.01895801
```

Némiképp robusztusabb szóródási mutató az interkvartilis terjedelem (Q2-Q3 közötti távolság) és az átlagos abszolút deviancia (*MAD*).

```
IQR(car_insur$CREDIT_SCORE, na.rm=TRUE)
```

```
[1] 0.2011203
```

```
mad(car_insur$CREDIT_SCORE, na.rm=TRUE)
```

```
[1] 0.1475353
```

8.1.4 Kvantilisek

A tiszta díj számításánál a kockázati prémium meghatározásához a felső kvantilisek (VaR-szerű mutatók) elemzése szükséges. A `quantile()` függvény alapértelmezetten csak a kvantilisek (Q1, Q2, Q3, Q4) számolja ki, ha eltérőt szeretnénk, akkor külön vektorban kell rögzíteni a kívánt értékeket.

```
quantile(car_insur$CREDIT_SCORE,
         probs = c(0.25, 0.5, 0.75, 0.95, 0.99),
         na.rm=TRUE) # Kvantilisek meghatározása
```

	25%	50%	75%	95%	99%
	0.4171913	0.5250328	0.6183117	0.7226941	0.7946105

8.1.5 Gyakoriságok

Gyakoriságot kizárólag diszkrét vagy kategorikus változók esetén tudunk számolni a már ismert `table()` paranccsal.

```
table(car_insur$EDUCATION)
```

high school	none	university
4157	1915	3928

A módusz számítására nincs külön függvény, mert az a gyakorisági tábla maximuma.

```
table(car_insur$SPEEDING_VIOLATIONS)
```

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
5028	1544	1161	830	530	319	188	140	75	49	50	30	20	12	5	8
16	17	18	19	22											
4	3	1	2	1											

```
# többféle módon is számolhatjuk a móduszt
```

```
max(table(car_insur$SPEEDING_VIOLATIONS)) # ez csak a maximum értéket adja meg, de nem mondja meg, hogy melyik
```

```
[1] 5028
```

```
# a legegyszerűbb, ha csökkenő sorrendbe rendezzük a sort() paranccsal
```

```
sort(table(car_insur$SPEEDING_VIOLATIONS), decreasing = TRUE)
```


0	1	2	3	4	5	6	7	8	10	9	11	12	13	15	14
5028	1544	1161	830	530	319	188	140	75	50	49	30	20	12	8	5
16	17	19	18	22											
4	3	2	1	1											

```
# mivel így az első elem lesz a módusz, akár kérhetjük csak ezt az elemet
sort(table(car_insur$SPEEDING_VIOLATIONS), decreasing = TRUE)[1]
```

```
0
5028
```

8.2 Alapvető statisztikai próbák

A hipotéziseinket statisztikai próbák segítségével ellenőrizhetjük. Fontos, hogy a H_0 -ról hozott döntésünk soha nem jelent megdönthetetlen bizonyítékot, csupán valamekkora bizonyosságot (konfidencia intervallum)! Ahhoz, hogy minél bizonyosabbak legyünk a próbánk eredményét illetően *megfelelő elemszám* és *véletlen, azonos eloszlásból származó minta* szükséges (lásd [Glivenko-Cantelli tétel](#)). Az ideális elemszám mítosza szerint létezik olyan minta, amely esetén már biztosan megfogalmazhatjuk állításunkat. Ez részben azon is múlik, hogy milyen próbát végzünk, milyen eloszlással dolgozunk. Általánosságban azonban az az igaz, hogy az $n + 1$ elemszámú minta mindig előnyösebb, mint az n elemszámú.

8.2.1 Hipotézisvizsgálat során elkövethető hibák

A hipotézis felállítása során a vizsgálatot végző rendszerint az első fajú hiba minimalizálására törekszik (H_0 -t elvetjük, holott igaz). Emellett nem szabad elfelejteni a másodfajú hibát sem (H_0 -t elfogadjuk, holott igaz)!

A populáció bármely paraméterére irányuló becslés esetén fennáll a hiba elkövetésének kockázata. Kétféle hibát követhetünk el:

- I. fajú hiba (α): H_0 -t elutasítjuk, miközben igaz.
- II. fajú hiba (β): H_0 -t elfogadjuk, miközben hamis.

Hipotéziseket rendszerint a populáció Θ paraméterére vonatkozóan szoktunk tenni, és rendszerint az attól való feltételezett $\hat{\Theta}$ eltérést (d) vetjük statisztikai próba alá:

$$P(|\hat{\Theta} - \Theta| > d < \alpha$$

Így az α azt a legkisebb valószínűséget jelenti, amely mellett még tévedhetünk.

8.2.2 Paraméteres próbák

8.2.2.1 Normalitás tesztek

A legtöbb próba paraméteres, azaz rendelkezünk kell valamilyen a populáció eloszlására vonatkozó feltétellel. Általában feltételezzük a normális eloszlást, erre vonatkozóan ugyanakkor nem igazán létezik megbízható próba. Az R bázisváltozata a `shapiro.test()` valamint a `ks.test` próbákat tartalmazza, amelyek elsősorban kis mintán alkalmazhatók, és nagyon érzékenyek. A `shapiro.test()` például legfeljebb 5000 megfigyelésen futtatható, ezért alkalmazzuk a `sample()` függvényt egy véletlen mintavételhez.

```
# Vizsgáljuk meg a credit_score változó normalitását  
  
shapiro.test(sample(car_insur$CREDIT_SCORE, 5000))
```

Shapiro-Wilk normality test

```
data:  sample(car_insur$CREDIT_SCORE, 5000)  
W = 0.99382, p-value = 4.718e-13
```

A Shapiro-próba null-hipotézise, hogy a vektor normális eloszlásból származik. Itt ezt el kell utasítanunk.

A Kolmogorov-Smirnov-próba már nagyobb mintán is elvégezhető, de nagyon érzékeny próbáról van szó, amely már a normalitástól való legkisebb eltérést is súlyosan bünteti. Előnye azonban, hogy szinte bármilyen eloszlást meg tudunk is vizsgálni. Mivel két *eloszlásfüggvény* összehasonlítása történik, ezért a referencia eloszlásnak is kumulatív eloszlásnak kell lennie. Ezt azonban elegendő szövegesen megfogalmazni a **pnorm** paraméterrel.

```
ks.test(car_insur$CREDIT_SCORE, "pnorm") # normális eloszlás
```

One-sample Kolmogorov-Smirnov test

```
data:  car_insur$CREDIT_SCORE  
D = 0.57035, p-value < 2.2e-16  
alternative hypothesis: two-sided
```

Jobban szokott működni a nem paraméteres [Jarque-Bera-próba](#), amely az eloszlás harmadik és negyedik momentumát hasonlítja a normális eloszlás momentumaihoz, ezáltal jóval robusztusabb

eredmény érhető el. A függvény a `tseries` csomagban található. Fontos, hogy nem szerepelhet a vektorban hiányzó adat, ezért azokat előbb az `na.omit` függvény segítségével hagyjuk figyelmen kívül.

```
library(tseries)
```

```
Registered S3 method overwritten by 'quantmod':  
  method      from  
as.zoo.data.frame zoo
```

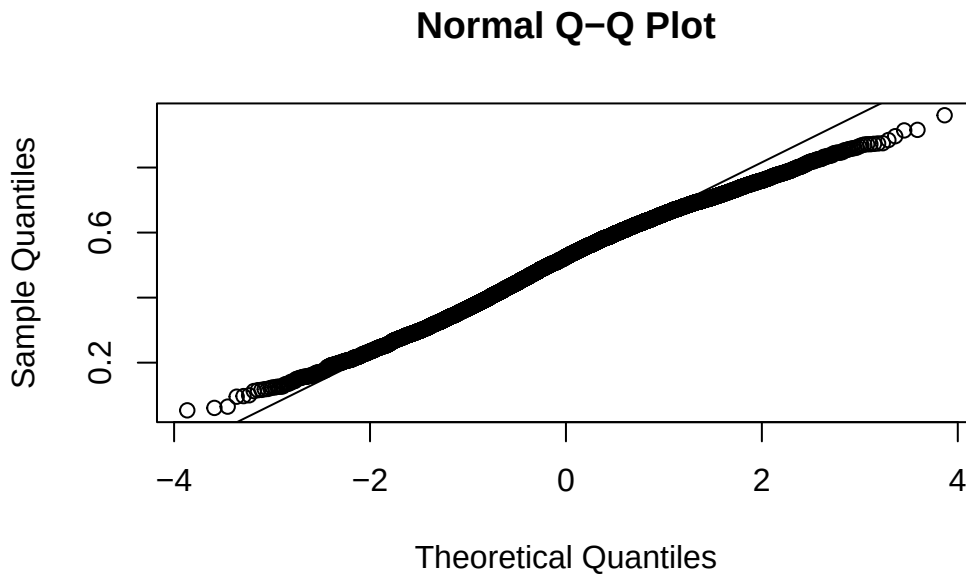
```
jarque.bera.test(na.omit(car_insur$CREDIT_SCORE))
```

Jarque Bera Test

```
data:  na.omit(car_insur$CREDIT_SCORE)  
X-squared = 139.75, df = 2, p-value < 2.2e-16
```

Sokszor nagyon hasznos az úgynevezett [QQ-plot](#) megjelenítés is, amely a tapasztalati eloszlás és a teoretikus eloszlás kvantiliseit vizsgálja. Nem tartozik hozzá semmilyen próba, a vizuális megjelenítés által szükséges döntést hozni. Ha a két eloszlás megegyezik, hogy a megfigyelések az azonossági ($x = y$) egyenesen helyezkednek el. A normális eloszlás vizsgálatára a `qqnorm()` az azonosság megjelenítésére a `qqline()` függvény alkalmazandó.

```
qqnorm(car_insur$CREDIT_SCORE)  
qqline(car_insur$CREDIT_SCORE)
```



Nincs olyan nagyon “nagy” eltérés a normális eloszlástól, inkább arról van szó, hogy mindkét végén erősen vastagodik az eloszlás széle.

i Eloszlásfüggvények (d, p, q, r család)

Az R egy egységesített eloszláskezelést folytat. Minden eloszláscsaládhoz 4 prefix tartozik, amelyekkel hivatkozhatunk az eloszlás típusára:

- **d (density)**: sűrűségfüggvény / tömegfüggvény értékek.
- **p (probability)**: kumulatív eloszlásfüggvény értékek.
- **q (quantile)**: kvantilis függvény (az eloszlásfüggvény inverze) értékek.
- **r (random)**: véletlen mintavétel.

Például kvantilis meghatározása 95%-os konfidenciaszinthez normál eloszlásnál:

```
qnorm(0.975)
```

```
[1] 1.959964
```

8.2.2.2 Várható értékre és szórásra vonatkozó próbák

Paraméteres próba lévén feltételezzük a normális eloszlást és a populáció varianciájának ismeretét. Ha mindkettő teljesül, akkor **Z-próbát** futtathatunk a $H_0 : \mu = \mu_0$ hipotézis ellenőrzésére. Ugyanakkor ez a valóságban nagyon ritkán van így, mivel a szórás rendre nem ismerjük. Ilyenkor a mintából becsüljük és **t-próbát** futtatunk. Például vizsgáljuk meg, hogy a férfiak és a nők által vezetett éves távolság. A R alapértelmezetten nem feltételezi a varianciák azonosságát, ezt azonban átállíthatjuk, ha nyomós indokunk van rá.

```
t.test(car_insur$ANNUAL_MILEAGE[car_insur$GENDER=="female"],
       car_insur$ANNUAL_MILEAGE[car_insur$GENDER=="male"],
       var.equal = FALSE)
```

Welch Two Sample t-test

```
data: car_insur$ANNUAL_MILEAGE[car_insur$GENDER == "female"] and car_insur$ANNUAL_MILEAGE[c
t = 1.5068, df = 9040.6, p-value = 0.1319
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -26.87515 205.49922
sample estimates:
mean of x mean of y
 11741.47  11652.15
```

A t-próba alapján a két nem által vezetett éves távolság átlaga nem különbözik szignifikánsan.

Hasonló analógián tesztelhetjük a szórás azonosságára vonatkozó hipotézisünket is a `var.test` függvénnyel.

```
var.test(car_insur$ANNUAL_MILEAGE[car_insur$GENDER=="female"],
         car_insur$ANNUAL_MILEAGE[car_insur$GENDER=="male"])
```

F test to compare two variances

```
data: car_insur$ANNUAL_MILEAGE[car_insur$GENDER == "female"] and car_insur$ANNUAL_MILEAGE[c
F = 1.0034, num df = 4540, denom df = 4501, p-value = 0.908
alternative hypothesis: true ratio of variances is not equal to 1
95 percent confidence interval:
 0.9466032 1.0636909
```

```
sample estimates:
ratio of variances
      1.003444
```

8.2.2.3 Arányokra vonatkozó próbák

A gyakorlatban gyakran előfordul, hogy arányokat szükséges összehasonlítani, például két csoport kárgyakoriságát. Fontos, hogy a próbák többsége kétutas, vagyis csak kettő csoport összehasonlítására van lehetőség. Többször összehasonlítás is lehetséges, ez azonban sokszor “kényelmetlen”, mert minden ugyanúgy egy hipotézist tudunk tesztelni, a próba azonban globális, azaz például $H_0 : \mu_1 = \mu_2 + \dots + \mu_m$.

Vizsgáljuk meg, hogy eltér-e a két nem kárgyakorisága! A `prop.test()` függvény több argumentumot kér. Nem szükséges kiszámolni az arányokat, hanem csak a “sikeres” próbálkozások számát kell megadni, valamint a mintanagyságot. A szűréshez használjuk most a `subset()` parancsot, amit nagyon hasonló a `dyplr` csomag `filter()` függvényéhez. Teszteljük a $H_0 : \mu_1 = \mu_2$ hipotézist.

```
prop.test(c(sum(subset(car_insur, car_insur$GENDER=="male")$OUTCOME), # férfiak kárgyakorisága
            sum(subset(car_insur, car_insur$GENDER=="female")$OUTCOME)), # nők kárgyakorisága
          c(nrow(subset(car_insur, car_insur$GENDER=="male")), # férfiak száma
            nrow(subset(car_insur, car_insur$GENDER=="female")) # nők száma
          )
```

2-sample test for equality of proportions with continuity correction

```
data:  c(sum(subset(car_insur, car_insur$GENDER == "male")$OUTCOME), sum(subset(car_insur, c
X-squared = 114.47, df = 1, p-value < 2.2e-16
alternative hypothesis: two.sided
95 percent confidence interval:
  0.08117319 0.11773401
sample estimates:
   prop 1    prop 2 
0.3631263 0.2636727
```

A kapott szignifikancia szint alapján bizonyosan elutasítjuk a null-hipotézist, de ez a 95%-os konfidencia intervallumból is látszik (nem tartalmazza a nullát).

8.3 Hipotézisek diszkrét változók esetén (univariáns)

A diszkrét adatok elemzése általában a kétutas kereszt táblákon keresztül történik. A leggyakrabban alkalmazott próba a `chisq.test()` függvény, aminek inputja egy kereszt tábla a `table()` függvény segítségével jön létre. A próba azon az elven működik, hogy ha egy kereszt tábla peremösszegeit ismerjük, akkor a két változó *függetlenségét* feltételezve becsülhetők a cella értékei. Ekkor a becslt (E) és a tapasztalati (O) értékek közötti különbséget ($H_0 : E - O = 0$) tudjuk tesztelni. A χ^2 próba null-hipotézise:

- H_0 : a két változó között **nincs összefüggés**.
- H_1 : a két változó között **van összefüggés**.

A példánkban vizsgáljuk meg, hogy van-e összefüggés az vezetési tapasztalt és a kárgyakoriság között.

```
table(car_insur$DRIVING_EXPERIENCE,  
       car_insur$OUTCOME) # gyakorisági tábla
```

	0	1
0-9y	1313	2217
10-19y	2512	787
20-29y	2010	109
30y+	1032	20

```
chisq.test(table(car_insur$DRIVING_EXPERIENCE,  
                 car_insur$OUTCOME))
```

Pearson's Chi-squared test

```
data:  table(car_insur$DRIVING_EXPERIENCE, car_insur$OUTCOME)  
X-squared = 2809.9, df = 3, p-value < 2.2e-16
```

Az eredmény alapján el kell utasítunk a null-hipotézist, és megállapíthatjuk, hogy valószínűleg van összefüggés aközött, hogy az ügyfélnek mekkora vezetési tapasztalat van és hány van-e káreseménye.

9 Adatvizualizáció és ggplot2 alapok

Az R-ben a vizualizáció szinte kizárólag a `ggplot2` elnevezésű csomagon keresztül történik. A lehetőségek tárháza szinte kimeríthetetlen, ugyanakkor kód és nem GUI alapú. A leggyakrabban használt vizualizációs lehetőségekről jó összefoglaló található az [R Graph Gallery](#) oldalon kódokkal és példákkal. Mindenképpen érdemes használni.

A `ggplot2` filozófiája szorosan összefonódik a `dplyr` csomaggal. Ugyan nem kötelező együtt használni a kettőt, de erősen ajánlott, mivel lerövidíthető, és jóval átláthatóbb kód készíthető.

A `ggplot2` “lelke” az `aes()` argumentum (“*aesthetics*”), amelyben az ábra tengelyeit, kategóriákat szükséges definiálni. Fontos, hogy az `aes()` kizárólag ezeket a beállításokat tartalmazza, ezután külön függvényben kell definiálni a diagrammtípust. `ggplot2` belül a grafikai függvényeket “+” operátor kapcsolja össze.

Természetesen alkalmazható az R saját grafikai motorja, ez azonban jóval “szegényesebb” megjelenítési lehetőségektől kínál, ráadásul jóval bonyolultabb kódolás mellett.

Egy ábrán véges dimenziószámot tudunk csak megjeleníteni. Két vagy három tengelyt tudunk definiálni, ezt színekkel kibővítve összesen 4 dimenzió megjelenítése a szokásos. Lehet ennél magasabb is, de akkor már kevésbé átlátható az ábra.

Jellemzően a következő lehetőségeink vannak:

- Egy numerikus változó esetén az *eloszlás és a sűrűség* jeleníthető meg.
- Egy diszkrét változó esetén a *gyakoriság* (tömegfüggvény) vizualizálható.
- Numerikus és numerikus változók között **korreláció** mérhető és ábrázolható.
- Kategorikus és kategorikus változók között *asszociáció* mérése és ábrázolása lehetséges.
- Kategorikus és numerikus adatok között *vegyes kapcsolat* ábrája készíthető el.

9.1 Eloszlások ábrázolása

A leggyakoribb ábrázolási lehetőség a **hisztogram** (diszkrét) és a **sűrűségfüggvény** (folytonos).

```
library(ggplot2)
require(dplyr)
```


Loading required package: dplyr

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

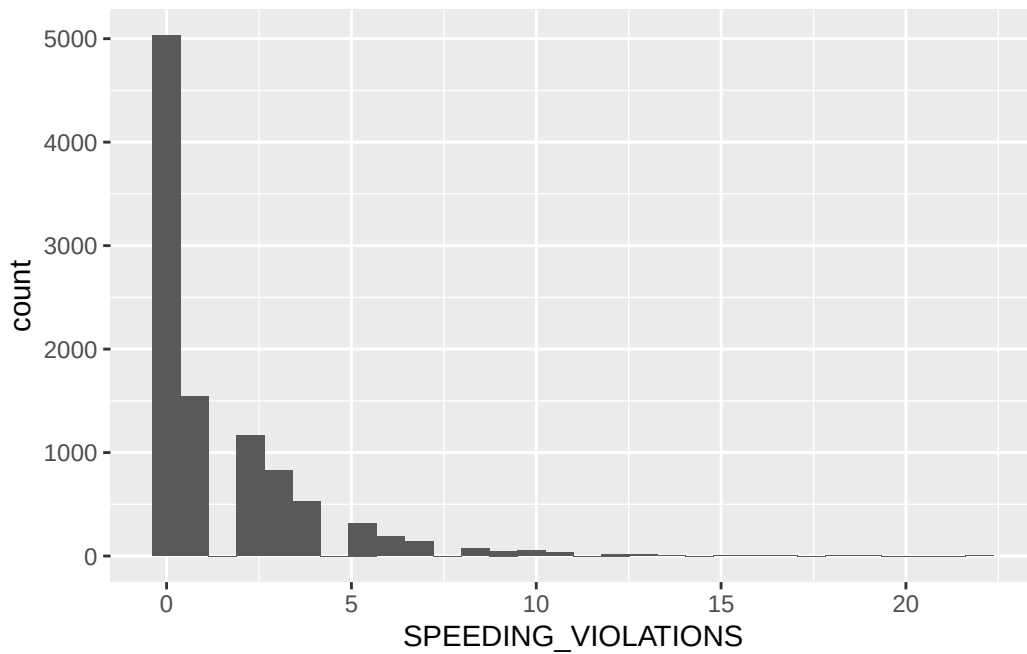
filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

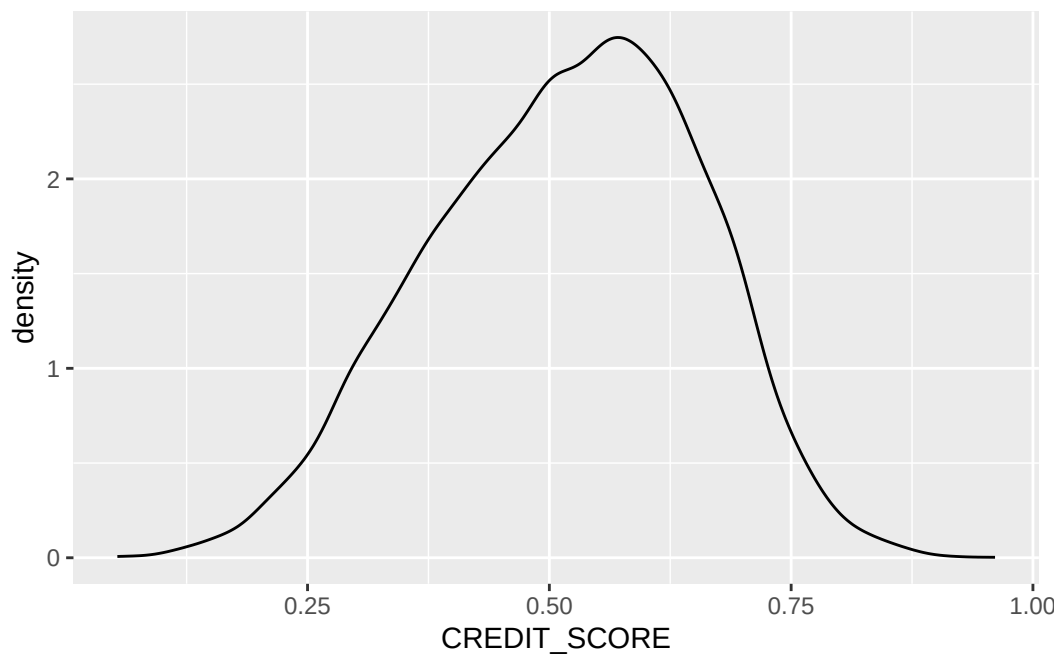
```
# hisztogramm - speedig violations  
car_insur %>%  
  ggplot(aes(SPEEDING_VIOLATIONS)) + geom_histogram()
```

`stat\_bin()` using `bins = 30`. Pick better value with `binwidth`.



```
# sűrűség - credit score
car_insur %>%
  ggplot(aes(x=CREDIT_SCORE)) + geom_density()
```

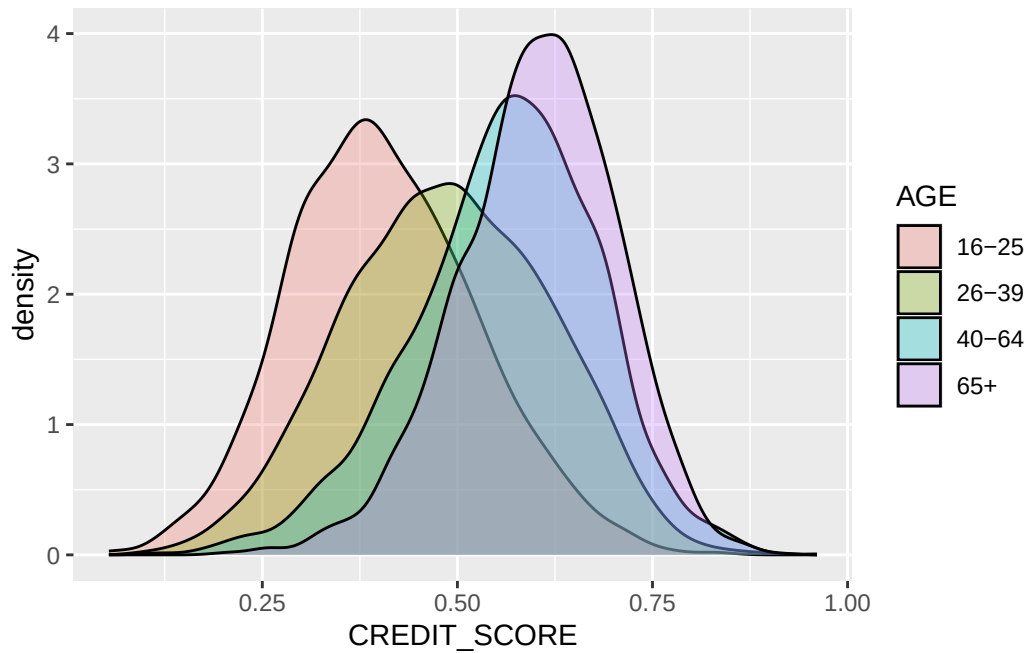
Warning: Removed 982 rows containing non-finite outside the scale range (`stat\_density()`).



A ggplot2 keretein belül mindig lehetőség van kategóriák szerinti bontásra.

```
car_insur %>%
  ggplot(aes(x=CREDIT_SCORE, group=AGE, fill = AGE)) + geom_density(alpha=0.3)
```

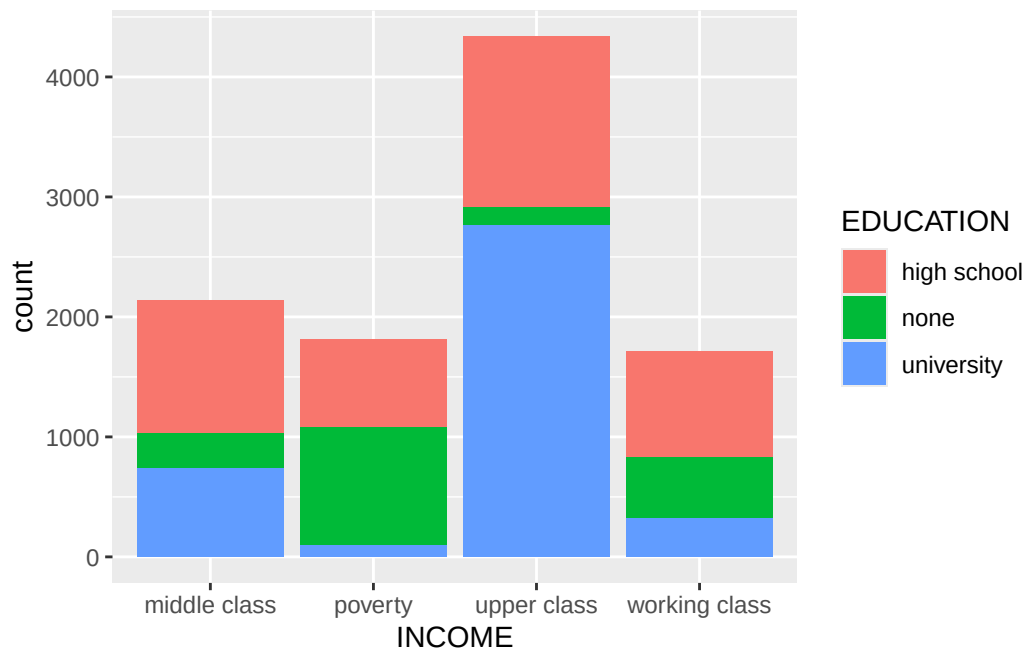
Warning: Removed 982 rows containing non-finite outside the scale range (`stat\_density()`).



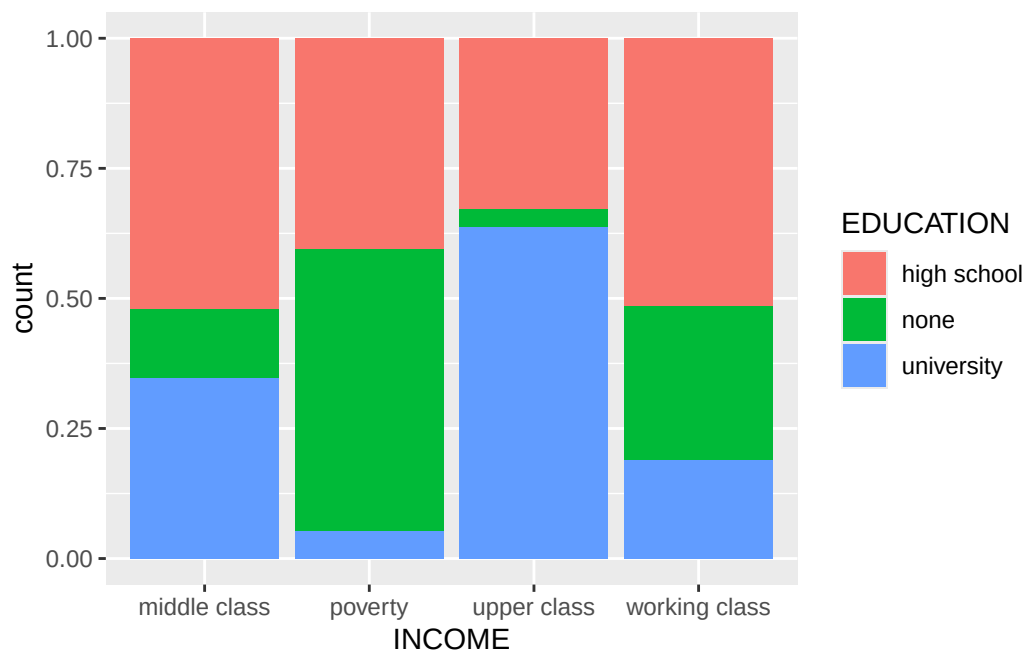
9.2 Asszociáció ábrázolása

Asszociációt két kategorikus változó között tudunk megjeleníteni úgy, hogy gyakorlatilag a gyakoriságokat tartalmazó kereszttáblát vizualizáljuk. Legyen a két változó az iskolai végzettség és az income. Vegyük észre, hogy a módszer nincs korlátozva a kategóriák száma által.

```
car_insur %>%
  ggplot(aes(x=INCOME, group=EDUCATION, fill=EDUCATION)) + geom_bar() # egyszerű oszlopdiagram
```



```
car_insur %>%
  ggplot(aes(x=INCOME, group=EDUCATION, fill=EDUCATION)) + geom_bar(position="fill") # 100% 1
```



9.3 Vegyes kapcsolat ábrázolása

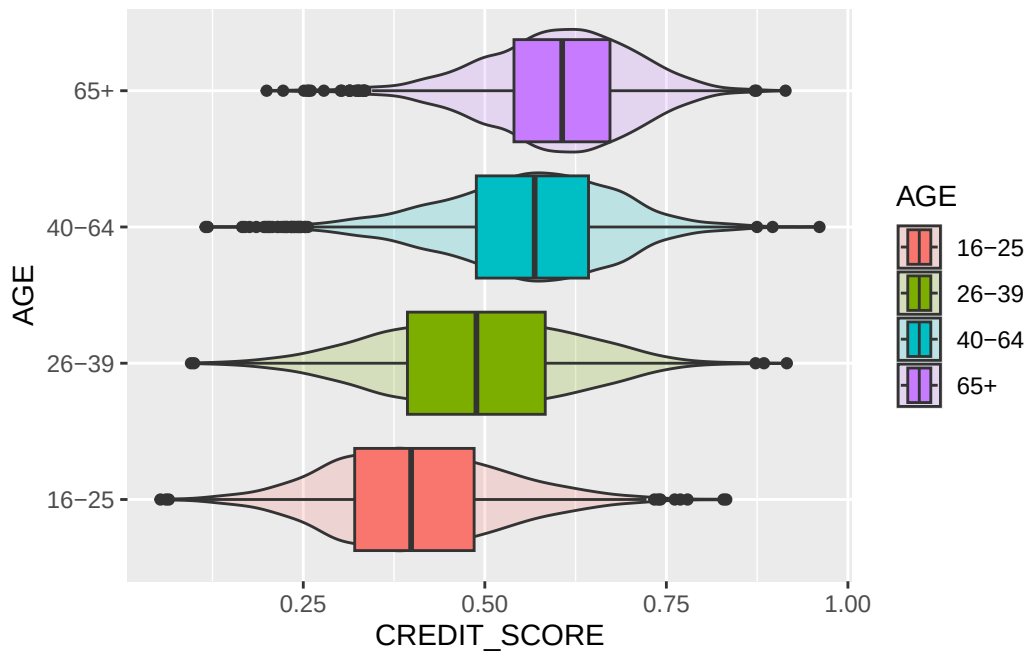
A vegyes kapcsolat vizualizációja nagyon hasonló az sűrűségfüggvények ábrázolásához, hiszen továbbra is folytonos változókat ábrázolunk, de külön kategóriánként tesszük mindezt. Az egy ábrán való megjelenít jelentősen megkönnyíti az elemzést.

Egy hasznos, több információt is megjelenítő ábra a hegedű diagramm, ami egyszerre képes elénk tárni a boxplot ábrát és a sűrűségfüggvény képét, mindezt összehasonlítható formában.

```
car_insur %>%  
  ggplot(aes(x=CREDIT_SCORE, y=AGE, fill=AGE)) + geom_violin(alpha=0.2) + geom_boxplot()
```

Warning: Removed 982 rows containing non-finite outside the scale range (`stat\_ydensity()`).

Warning: Removed 982 rows containing non-finite outside the scale range (`stat\_boxplot()`).

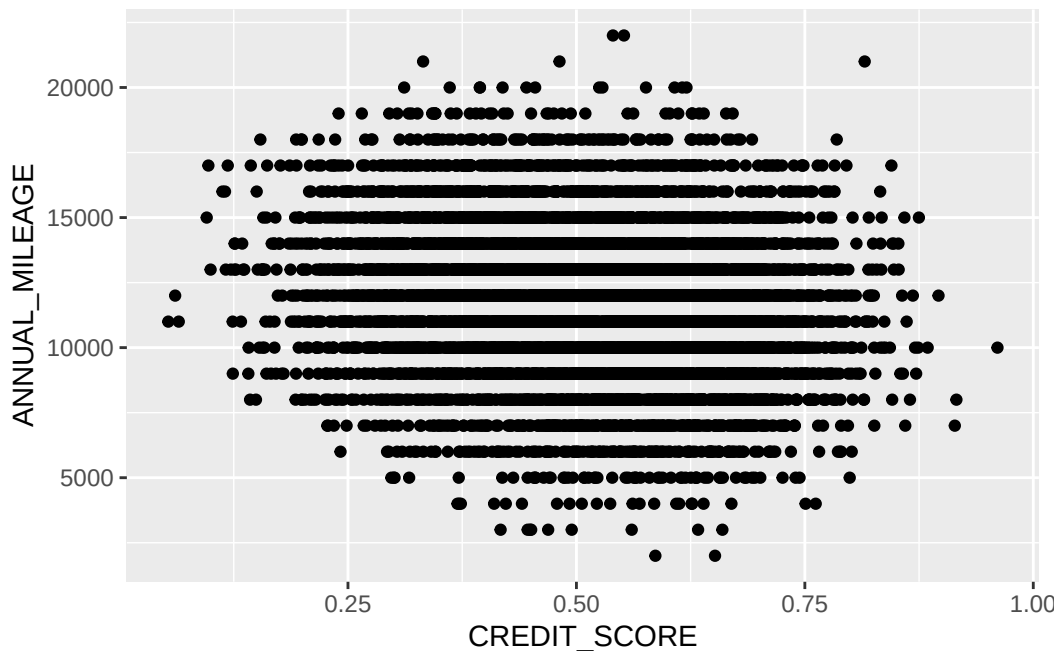


9.4 Korreláció ábrázolása

A kétváltozós ábrák legtöbbször két folytonos változó terében mutatják meg az adatok sűrűsödését. Erre több lehetőség is kínálkozik, a leggyakrabban a pontdiagramm (felhődiagramm) alkalmazott. Sajnos itt csak két valóban numerikus változónk van, amely összehasonlítása nem feltétlenül értelmes. Ábrázoljuk a *credit_score* és az *annual_milage* változókat pontdiagrammon.

```
car_insur %>%  
  ggplot(aes(x=CREDIT_SCORE, y=ANNUAL_MILEAGE)) + geom_point()
```

Warning: Removed 1851 rows containing missing values or values outside the scale range (`geom\_point()`).



Mindig van lehetőségünk valamilyen trendvonal felvételére, legyen az lineáris vagy simított, súlyozott (LOESS).

```
car_insur %>%  
  ggplot(aes(x=CREDIT_SCORE, y=ANNUAL_MILEAGE)) + geom_point() + geom_smooth(method = "lm",  
    geom_smooth(method="loess", color="blue") # simított, súlyozott
```

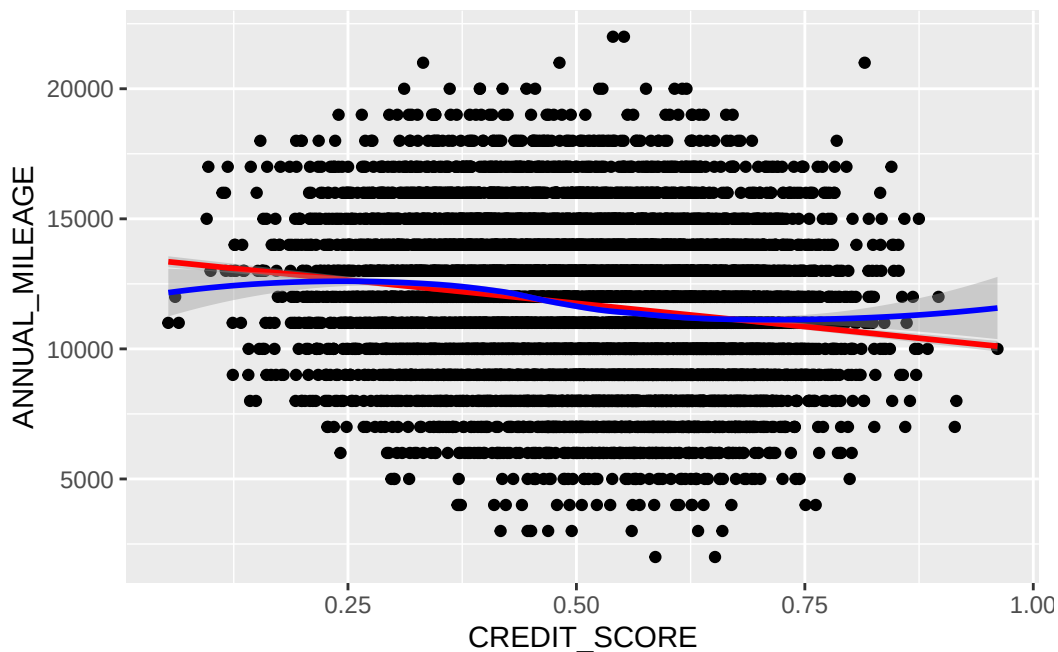
```
`geom_smooth()` using formula = 'y ~ x'
```

```
Warning: Removed 1851 rows containing non-finite outside the scale range
(`stat_smooth()`).
```

```
`geom_smooth()` using formula = 'y ~ x'
```

```
Warning: Removed 1851 rows containing non-finite outside the scale range
(`stat_smooth()`).
```

```
Warning: Removed 1851 rows containing missing values or values outside the scale range
(`geom_point()`).
```



Néha látványosabb hőtérképen megjeleníteni az adatokat, mert így még egy dimenziót gond nélkül tudunk ábrázolni.

```
car_insur %>% ggplot(aes(x=ANNUAL_MILEAGE, y=SPEEDING_VIOLATIONS, fill=CREDIT_SCORE)) + geom.
```

```
Warning: Removed 957 rows containing missing values or values outside the scale range
(`geom_tile()`).
```

